DEPARTMENT OF COMPUTER SCIENCES • SYSTEMS GROUP

Consensus Protocols in Distributed Storage

Formal Mechanisms of Raft, Multi-Paxos, and Viewstamped Replication

PRESENTER

Prof. Alan Kay

DATE

Lecture 08 • Fall 2026

FOUNDATIONS

The Consensus Problem: Formal Bounds

State Machine Replication under Asynchronous Network Assumptions

- **FLP Impossibility Result**: Deterministic asynchronous consensus is impossible in the presence of even a single unannounced crash failure.
- **Partially Synchronous Models**: Practical systems circumvent FLP by introducing randomized timers or weak synchrony Δ guarantees.
- **State Machine Replication (SMR)**: All replicas execute identical sequences of deterministic transitions from an identical initial state S_0.
- **Safety Over Liveness**: Distributed algorithms guarantee linearizability even during unbounded network partitions.

> KEY TAKEAWAY
>
> *Consensus does not guarantee continuous availability during partitions; it guarantees strict non-divergence of committed logs.*

PROTOCOL MECHANICS

Raft: Quorum-Based Log Replication

Normal operation flow and write replication under leader authority

- **Client Proposal**: Client issues mutation $v = 42$ exclusively to the established leader of term T.
- **AppendEntries RPC**: Leader appends proposal locally, then broadcasts parallel RPCs to all registered peers.
- **Commit Threshold**: Once replicated across a majority quorum $\lfloor \frac{N}{2} \rfloor + 1$, the entry is permanently committed.
- **State Machine Apply**: Committed index advances; leader responds to client and signals followers on subsequent heartbeats.

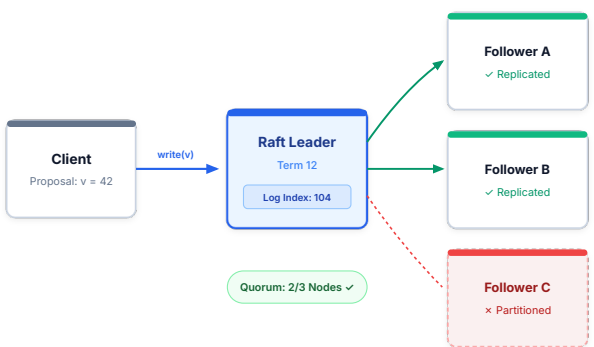


Figure 1: Leader appends entries to local log before dispatching AppendEntries RPCs.

ARCHITECTURAL TRADE-OFFS

Comparative Analysis: Raft vs Multi-Paxos

Deconstructing architectural simplicity versus operational throughput

Raft (Strong Leader Approach)

- **Log Discrepancy Prevention**: Log entries only flow unidirectionally from Leader to Followers.
- **Understandability Metric**: Formally designed to eliminate ambiguous dual-leader overlap states.
- **Leader Bottleneck**: Single leader node processes 100% of write coordination and sequencing.

Multi-Paxos (Decentralized Consensus)

- **Flexible Roles**: Any replica can propose; phase-1 preparation can be amortized across instances.
- **Complex Invariant Inversion**: Gap filling and out-of-order log hole resolution introduces edge cases.
- **High Throughput Ceiling**: Accommodates multi-leader topologies (e.g. Egalitarian Paxos).

Synthesis: Raft reduces operational bugs in production control planes (etcd, Consul), whereas Multi-Paxos variants excel in geo-replicated active-active databases (Spanner, CockroachDB).

EMPIRICAL BENCHMARKS

Empirical Performance Under Cluster Degradation

14.2k ops/sec

Peak Write Throughput

3-node NVMe Raft cluster with fsync batching

3.1 ms

P99 Commit Latency

Local area gigabit network interconnect

180 ms

Leader Failover Time

Heartbeat interval 50ms, timeout 150-300ms

0 anomalies

Linearizability Violations

Jepsen partition injection suite verification

> **Empirical Finding**: Randomized follower election timeouts (150–300ms) prevent persistent split-vote livelocks with >99.7% statistical probability.

IMPLEMENTATION

Leader Election Timeout State Transition

```python
def handle_election_timeout(self):
    if self.state == NodeState.LEADER:
        return

    self.current_term += 1
    self.state = NodeState.CANDIDATE
    self.voted_for = self.node_id
    self.votes_received = {self.node_id}
    self.reset_timer(random_election_timeout())

    # Dispatch RequestVote RPC to all peers
    for peer in self.peers:
        self.send_request_vote(
            peer_id=peer,
            term=self.current_term,
            candidate_id=self.node_id,
            last_log_idx=self.log.last_index(),
            last_log_term=self.log.last_term()
        )
```

- **Term Incrementation**: Transitioning to Candidate immediately advances monotonic clock `current_term`.
- **Self-Voting Invariant**: Candidate casts its own vote before dispatching network requests.
- **Log Up-to-Date Rule**: Peers reject `RequestVote` if candidate's log is less complete than recipient's.